

50+ INFOSYS CODING QUESTIONS WITH PYTHON SOLUTIONS

Created by SYNTAX ERROR | Abhishek Rathor

Each solution includes:

- Problem Statement
- Time & Space Complexity
- Test Cases

===== SECTION 1: ARRAYS & LISTS =====

Question 1: Find the Largest Element in a List

"""

Problem: Given a list of integers, find and return the largest element.

Approach: Iterate through the list and keep track of maximum element.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

"""

```
def find_largest(arr):
```

```
    if not arr:
```

```
        return None
```

```
    max_val = arr[0]
```

```
    for num in arr:
```

```
        if num > max_val:
```

```
            max_val = num
```

```
    return max_val
```

Alternative using built-in

```
def find_largest_v2(arr):
```

```
    return max(arr) if arr else None
```

```
print("Question 1: Find Largest Element")
test1 = [3, 7, 2, 9, 1, 5]
print(f"Input: {test1}")
print(f"Output: {find_largest(test1)}")
print(f"Expected: 9\n")
```

Question 2: Reverse a List

```
"""
```

Problem: Reverse the given list in-place and also return a new reversed list.

Approach: Use two-pointer technique or Python slicing.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ for in-place, $O(n)$ for new list

```
"""
```

```
def reverse_list_inplace(arr):
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr
```

```
def reverse_list(arr):
    return arr[::-1]
```

```
print("Question 2: Reverse a List")
test2 = [1, 2, 3, 4, 5]
print(f"Input: {test2}")
print(f"Output: {reverse_list(test2.copy())}")
```

```
print(f'Expected: [5, 4, 3, 2, 1]\n')
```

Question 3: Check if String is Palindrome

```
"""
```

Problem: Check if a given string reads the same forwards and backwards.

Approach: Compare string with its reverse or use two pointers.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
"""
```

```
def is_palindrome(s):
```

```
    # Remove spaces and convert to lowercase
```

```
    s = s.replace(" ", "").lower()
```

```
    left, right = 0, len(s) - 1
```

```
    while left < right:
```

```
        if s[left] != s[right]:
```

```
            return False
```

```
        left += 1
```

```
        right -= 1
```

```
    return True
```

```
def is_palindrome_v2(s):
```

```
    s = s.replace(" ", "").lower()
```

```
    return s == s[::-1]
```

```
print("Question 3: Check Palindrome")
```

```
test3 = "A man a plan a canal Panama"
```

```
print(f'Input: '{test3}')
```

```
print(f'Output: {is_palindrome(test3)}')
```

```
print(f'Expected: True\n')
```

Question 4: Find Second Largest Element

"""

Problem: Find the second largest element in the list.

Approach: Keep track of first and second maximum while iterating.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

"""

```
def second_largest(arr):  
    if len(arr) < 2:  
        return None  
  
    first = second = float('-inf')  
  
    for num in arr:  
        if num > first:  
            second = first  
            first = num  
        elif num > second and num != first:  
            second = num  
  
    return second if second != float('-inf') else None  
  
print("Question 4: Second Largest Element")  
test4 = [12, 35, 1, 10, 34, 1]  
print(f"Input: {test4}")  
print(f"Output: {second_largest(test4)}")  
print(f"Expected: 34\n")
```

Question 5: Remove Duplicates from List

"""

Problem: Remove duplicate elements while maintaining original order.

Approach: Use dict to maintain insertion order (Python 3.7+).

Time Complexity: $O(n)$

Space Complexity: $O(n)$

"""

```
def remove_duplicates(arr):
```

```
    return list(dict.fromkeys(arr))
```

```
def remove_duplicates_v2(arr):
```

```
    seen = set()
```

```
    result = []
```

```
    for num in arr:
```

```
        if num not in seen:
```

```
            seen.add(num)
```

```
            result.append(num)
```

```
    return result
```

```
print("Question 5: Remove Duplicates")
```

```
test5 = [1, 2, 2, 3, 4, 4, 5, 1]
```

```
print(f"Input: {test5}")
```

```
print(f"Output: {remove_duplicates(test5)}")
```

```
print(f"Expected: [1, 2, 3, 4, 5]\n")
```

Question 6: Find Missing Number (1 to n)

"""

Problem: Given array of n-1 integers in range 1 to n, find missing number.

Approach: Use sum formula $n*(n+1)/2$ and subtract array sum.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
"""
```

```
def find_missing(arr, n):  
    expected_sum = n * (n + 1) // 2  
    actual_sum = sum(arr)  
    return expected_sum - actual_sum
```

```
print("Question 6: Find Missing Number")
```

```
test6 = [1, 2, 4, 5, 6]
```

```
n = 6
```

```
print(f"Input: {test6}, n={n}")
```

```
print(f"Output: {find_missing(test6, n)}")
```

```
print(f"Expected: 3\n")
```

Question 7: Rotate List by K Positions

```
"""
```

Problem: Rotate list to the right by k positions.

Approach: Slice and concatenate: last k elements + first n-k elements.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
"""
```

```
def rotate_list(arr, k):  
    if not arr:  
        return arr  
    k = k % len(arr) # Handle k > len(arr)  
    return arr[-k:] + arr[:-k]
```

```
def rotate_list_left(arr, k):
```

```

    if not arr:
        return arr

    k = k % len(arr)

    return arr[k:] + arr[:k]

print("Question 7: Rotate List")
test7 = [1, 2, 3, 4, 5, 6, 7]
k = 3
print(f"Input: {test7}, k={k}")
print(f"Output (right): {rotate_list(test7, k)}")
print(f"Expected: [5, 6, 7, 1, 2, 3, 4]\n")

```

Question 8: Count Vowels in String

"""

Problem: Count the number of vowels in a string.

Approach: Iterate and check each character against vowel set.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

"""

```

def count_vowels(s):
    vowels = set('aeiouAEIOU')
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

def count_vowels_v2(s):
    return sum(1 for char in s if char.lower() in 'aeiou')

```

```

print("Question 8: Count Vowels")
test8 = "Hello World"
print(f'Input: '{test8}''')
print(f'Output: {count_vowels(test8)}')
print(f'Expected: 3\n')

```

Question 9: Check if Two Strings are Anagrams

"""

Problem: Check if two strings contain same characters with same frequencies.

Approach: Sort both strings and compare, or use character frequency count.

Time Complexity: $O(n \log n)$ for sorting, $O(n)$ for frequency count

Space Complexity: $O(n)$

"""

```

def are_anagrams(s1, s2):
    if len(s1) != len(s2):
        return False
    return sorted(s1.lower()) == sorted(s2.lower())

def are_anagrams_v2(s1, s2):
    from collections import Counter
    return Counter(s1.lower()) == Counter(s2.lower())

```

```

print("Question 9: Check Anagrams")
test9a, test9b = "listen", "silent"
print(f'Input: '{test9a}', '{test9b}''')
print(f'Output: {are_anagrams(test9a, test9b)}')
print(f'Expected: True\n')

```

Question 10: Find Intersection of Two Lists

"""

Problem: Find common elements between two lists.

Approach: Convert to sets and use set intersection.

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

"""

```
def find_intersection(arr1, arr2):  
    return list(set(arr1) & set(arr2))
```

```
def find_intersection_sorted(arr1, arr2):
```

```
    # If arrays are sorted
```

```
    result = []
```

```
    i = j = 0
```

```
    while i < len(arr1) and j < len(arr2):
```

```
        if arr1[i] < arr2[j]:
```

```
            i += 1
```

```
        elif arr1[i] > arr2[j]:
```

```
            j += 1
```

```
        else:
```

```
            result.append(arr1[i])
```

```
            i += 1
```

```
            j += 1
```

```
    return result
```

```
print("Question 10: Find Intersection")
```

```
test10a = [1, 2, 3, 4, 5]
```

```
test10b = [3, 4, 5, 6, 7]
```

```
print(f"Input: {test10a}, {test10b}")
```

```
print(f"Output: {sorted(find_intersection(test10a, test10b))}")
```

```
print(f"Expected: [3, 4, 5]\n")
```

===== SECTION 2: SEARCHING & SORTING =====

Question 11: Binary Search

"""

Problem: Search for target element in sorted array.

Approach: Divide and conquer - compare with middle element.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

"""

```
def binary_search(arr, target):
```

```
    left, right = 0, len(arr) - 1
```

```
    while left <= right:
```

```
        mid = left + (right - left) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return -1
```

```
def binary_search_recursive(arr, target, left, right):
```

```
    if left > right:
```

```
        return -1
```

```
    mid = left + (right - left) // 2
```

```
    if arr[mid] == target:
```

```

        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)

print("Question 11: Binary Search")
test11 = [1, 3, 5, 7, 9, 11, 13, 15]
target = 7
print(f"Input: {test11}, target={target}")
print(f"Output: {binary_search(test11, target)}")
print(f"Expected: 3\n")

```

Question 12: Bubble Sort

```

"""

```

Problem: Sort array using bubble sort algorithm.

Approach: Repeatedly swap adjacent elements if in wrong order.

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best

Space Complexity: $O(1)$

```

"""

```

```

def bubble_sort(arr):
    n = len(arr)
    arr = arr.copy()

    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

```

```
swapped = True
```

```
if not swapped: # Optimization
```

```
    break
```

```
return arr
```

```
print("Question 12: Bubble Sort")
```

```
test12 = [64, 34, 25, 12, 22, 11, 90]
```

```
print(f"Input: {test12}")
```

```
print(f"Output: {bubble_sort(test12)}")
```

```
print(f"Expected: [11, 12, 22, 25, 34, 64, 90]\n")
```

Question 13: Selection Sort

```
"""
```

Problem: Sort array using selection sort algorithm.

Approach: Select minimum element and place it at beginning.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

```
"""
```

```
def selection_sort(arr):
```

```
    arr = arr.copy()
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_idx = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_idx]:
```

```
                min_idx = j
```

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```
return arr
```

```
print("Question 13: Selection Sort")
```

```
test13 = [64, 25, 12, 22, 11]
```

```
print(f'Input: {test13}')
```

```
print(f'Output: {selection_sort(test13)}')
```

```
print(f'Expected: [11, 12, 22, 25, 64]\n')
```

Question 14: Insertion Sort

```
"""
```

Problem: Sort array using insertion sort algorithm.

Approach: Build sorted array one element at a time.

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best

Space Complexity: $O(1)$

```
"""
```

```
def insertion_sort(arr):
```

```
    arr = arr.copy()
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
```

```
            arr[j + 1] = arr[j]
```

```
            j -= 1
```

```
arr[j + 1] = key
```

```
return arr
```

```
print("Question 14: Insertion Sort")
test14 = [12, 11, 13, 5, 6]
print(f"Input: {test14}")
print(f"Output: {insertion_sort(test14)}")
print(f"Expected: [5, 6, 11, 12, 13]\n")
```

Question 15: Find First and Last Position

```
"""
```

Problem: Find first and last occurrence of target in array.

Approach: Linear scan or binary search for first and last.

Time Complexity: O(n) linear, O(log n) binary search

Space Complexity: O(1)

```
"""
```

```
def find_first_last(arr, target):
```

```
    first = last = -1
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == target:
```

```
            if first == -1:
```

```
                first = i
```

```
                last = i
```

```
    return first, last
```

```
print("Question 15: First and Last Position")
```

```
test15 = [1, 2, 3, 3, 3, 4, 5]
target = 3
print(f'Input: {test15}, target={target}')
print(f'Output: {find_first_last(test15, target)}')
print(f'Expected: (2, 4)\n')
```

===== SECTION 3: MATHEMATICAL PROBLEMS =====

Question 16: Check if Number is Prime

```
"""
```

Problem: Determine if a number is prime.

Approach: Check divisibility up to $\sqrt{n}$.

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$

```
"""
```

```
def is_prime(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    if n <= 3:
```

```
        return True
```

```
    if n % 2 == 0 or n % 3 == 0:
```

```
        return False
```

```
    i = 5
```

```
    while i * i <= n:
```

```
        if n % i == 0 or n % (i + 2) == 0:
```

```
            return False
```

```
        i += 6
```

```
    return True
```

```
print("Question 16: Check Prime Number")
test16 = 29
print(f"Input: {test16}")
print(f"Output: {is_prime(test16)}")
print(f"Expected: True\n")
```

Question 17: Factorial

```
"""
```

Problem: Calculate factorial of a number.

Approach: Recursive or iterative multiplication.

Time Complexity: $O(n)$

Space Complexity: $O(n)$ recursive, $O(1)$ iterative

```
"""
```

```
def factorial(n):
```

```
    if n < 0:
```

```
        return None
```

```
    if n <= 1:
```

```
        return 1
```

```
    return n * factorial(n - 1)
```

```
def factorial_iterative(n):
```

```
    if n < 0:
```

```
        return None
```

```
    result = 1
```

```
    for i in range(2, n + 1):
```

```
        result *= i
```

```
    return result
```



```
print("Question 17: Factorial")
test17 = 5
print(f'Input: {test17}')
print(f'Output: {factorial(test17)}')
print(f'Expected: 120\n')
```

Question 18: Fibonacci Number

```
"""
```

Problem: Find nth Fibonacci number.

Approach: Recursion with memoization or iterative.

Time Complexity: $O(n)$ with memoization, $O(2^n)$ without

Space Complexity: $O(n)$

```
"""
```

```
def fibonacci(n, memo={}):
```

```
    if n in memo:
```

```
        return memo[n]
```

```
    if n <= 1:
```

```
        return n
```

```
    memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo)
```

```
    return memo[n]
```

```
def fibonacci_iterative(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    a, b = 0, 1
```

```
    for _ in range(2, n + 1):
```

```
        a, b = b, a + b
```

```
return b
```

```
print("Question 18: Fibonacci Number")
```

```
test18 = 10
```

```
print(f"Input: {test18}")
```

```
print(f"Output: {fibonacci(test18)}")
```

```
print(f"Expected: 55\n")
```

Question 19: GCD (Greatest Common Divisor)

```
"""
```

Problem: Find GCD of two numbers.

Approach: Euclidean algorithm.

Time Complexity: $O(\log \min(a,b))$

Space Complexity: $O(1)$

```
"""
```

```
def gcd(a, b):
```

```
    while b:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def gcd_recursive(a, b):
```

```
    if b == 0:
```

```
        return a
```

```
    return gcd_recursive(b, a % b)
```

```
print("Question 19: GCD")
```

```
test19a, test19b = 48, 18
```

```
print(f"Input: {test19a}, {test19b}")
```

```
print(f'Output: {gcd(test19a, test19b)}')
print(f'Expected: 6\n')
```

Question 20: LCM (Least Common Multiple)

"""

Problem: Find LCM of two numbers.

Approach: Use formula $LCM(a,b) = (a*b)/GCD(a,b)$.

Time Complexity: $O(\log \min(a,b))$

Space Complexity: $O(1)$

"""

```
def lcm(a, b):
    return abs(a * b) // gcd(a, b)
```

```
print("Question 20: LCM")
test20a, test20b = 12, 15
print(f'Input: {test20a}, {test20b}')
print(f'Output: {lcm(test20a, test20b)}')
print(f'Expected: 60\n')
```

Question 21: Armstrong Number

"""

Problem: Check if number equals sum of its digits raised to power of digit count.

Approach: Extract digits, calculate power sum, compare.

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

"""

```
def is_armstrong(n):
    num_str = str(abs(n))
```

```

power = len(num_str)
total = sum(int(digit) ** power for digit in num_str)
return total == abs(n)

print("Question 21: Armstrong Number")
test21 = 153
print(f"Input: {test21}")
print(f"Output: {is_armstrong(test21)}")
print(f"Calculation:  $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ ")
print(f"Expected: True\n")

```

Question 22: Perfect Number

"""

Problem: Check if number equals sum of its proper divisors.

Approach: Find all divisors except number itself and sum them.

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$

"""

```

def is_perfect(n):
    if n <= 1:
        return False

    divisor_sum = 1
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            divisor_sum += i
            if i != n // i:
                divisor_sum += n // i

```

```
return divisor_sum == n
```

```
print("Question 22: Perfect Number")
```

```
test22 = 28
```

```
print(f"Input: {test22}")
```

```
print(f"Output: {is_perfect(test22)}")
```

```
print(f"Divisors: 1, 2, 4, 7, 14 → Sum = 28")
```

```
print(f"Expected: True\n")
```

Question 23: Sum of Digits

```
"""
```

Problem: Calculate sum of all digits in a number.

Approach: Extract digits one by one and add.

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

```
"""
```

```
def sum_of_digits(n):
```

```
    return sum(int(digit) for digit in str(abs(n)))
```

```
def sum_of_digits_v2(n):
```

```
    n = abs(n)
```

```
    total = 0
```

```
    while n > 0:
```

```
        total += n % 10
```

```
        n //= 10
```

```
    return total
```

```
print("Question 23: Sum of Digits")
```

```
test23 = 12345
```

```
print(f'Input: {test23}')
print(f'Output: {sum_of_digits(test23)}')
print(f'Expected: 15\n')
```

Question 24: Reverse a Number

"""

Problem: Reverse the digits of a number.

Approach: Extract digits from right and build reversed number.

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

"""

```
def reverse_number(n):
    sign = -1 if n < 0 else 1
    n = abs(n)
    reversed_num = 0

    while n > 0:
        reversed_num = reversed_num * 10 + n % 10
        n //= 10

    return sign * reversed_num
```

```
def reverse_number_v2(n):
    sign = -1 if n < 0 else 1
    return sign * int(str(abs(n))[::-1])
```

```
print("Question 24: Reverse Number")
test24 = 12345
print(f'Input: {test24}')
```

```
print(f'Output: {reverse_number(test24)}')
print(f'Expected: 54321\n')
```

Question 25: Palindrome Number

"""

Problem: Check if number reads same forwards and backwards.

Approach: Reverse number and compare with original.

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

"""

```
def is_palindrome_number(n):
    if n < 0:
        return False
    return n == reverse_number(n)
```

```
print("Question 25: Palindrome Number")
test25 = 12321
print(f'Input: {test25}')
print(f'Output: {is_palindrome_number(test25)}')
print(f'Expected: True\n')
```

===== SECTION 4: PATTERN PRINTING =====

Question 26: Right Triangle Pattern

"""

Problem: Print right triangle pattern with stars.

```

*

* *

* * *

* * * *

"""

def print_right_triangle(n):
    for i in range(1, n + 1):
        print('* ' * i)

print("Question 26: Right Triangle Pattern (n=4)")
print_right_triangle(4)
print()

```

Question 27: Pyramid Pattern

```

"""

Problem: Print pyramid pattern with stars.

*

***

*****

*****

"""

def print_pyramid(n):
    for i in range(1, n + 1):
        spaces = ' ' * (n - i)
        stars = '*' * (2 * i - 1)
        print(spaces + stars)

print("Question 27: Pyramid Pattern (n=4)")
print_pyramid(4)

```



```
print()
```

Question 28: Number Pyramid

```
"""
```

Problem: Print number pyramid pattern.

```
1
```

```
1 2
```

```
1 2 3
```

```
1 2 3 4
```

```
"""
```

```
def print_number_pyramid(n):
```

```
    for i in range(1, n + 1):
```

```
        for j in range(1, i + 1):
```

```
            print(j, end=' ')
```

```
        print()
```

```
print("Question 28: Number Pyramid (n=4)")
```

```
print_number_pyramid(4)
```

```
print()
```

Question 29: Diamond Pattern

```
"""
```

Problem: Print diamond pattern with stars.

```
"""
```

```
def print_diamond(n):
```

```
    # Upper half
```

```
    for i in range(1, n + 1):
```

```
        print(' ' * (n - i) + '*' * (2 * i - 1))
```

```
    # Lower half
```

```

    for i in range(n - 1, 0, -1):
        print(' ' * (n - i) + '*' * (2 * i - 1))

print("Question 29: Diamond Pattern (n=4)")
print_diamond(4)
print()

```

Question 30: Hollow Square

```

"""
Problem: Print hollow square pattern with stars.
"""

def print_hollow_square(n):
    for i in range(n):
        if i == 0 or i == n - 1:
            print('* ' * n)
        else:
            print('* ' + ' ' * (n - 2) + '*')

print("Question 30: Hollow Square (n=5)")
print_hollow_square(5)
print()

```

===== SECTION 5: RECURSION =====

Question 31: Power Function

```
"""
```

Problem: Calculate $\text{base}^{\text{exponent}}$ using recursion.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
"""
```

```
def power(base, exp):
```

```
    if exp == 0:
```

```
        return 1
```

```
    if exp < 0:
```

```
        return 1 / power(base, -exp)
```

```
    return base * power(base, exp - 1)
```

```
def power_optimized(base, exp):
```

```
    # Fast exponentiation
```

```
    if exp == 0:
```

```
        return 1
```

```
    if exp < 0:
```

```
        return 1 / power_optimized(base, -exp)
```

```
    half = power_optimized(base, exp // 2)
```

```
    if exp % 2 == 0:
```

```
        return half * half
```

```
    else:
```

```
        return base * half * half
```

```
print("Question 31: Power Function")
```

```
test31_base, test31_exp = 2, 10
```

```
print(f"Input: base={test31_base}, exp={test31_exp}")
```

```
print(f"Output: {power(test31_base, test31_exp)}")
```

```
print(f"Expected: 1024\n")
```

Question 32: Sum of Natural Numbers

"""

Problem: Calculate sum of first n natural numbers using recursion.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

"""

```
def sum_natural(n):
```

```
    if n <= 0:
```

```
        return 0
```

```
    return n + sum_natural(n - 1)
```

```
def sum_natural_formula(n):
```

```
    # Direct formula:  $n*(n+1)/2$ 
```

```
    return n * (n + 1) // 2
```

```
print("Question 32: Sum of Natural Numbers")
```

```
test32 = 10
```

```
print(f"Input: {test32}")
```

```
print(f"Output: {sum_natural(test32)}")
```

```
print(f"Expected: 55\n")
```

Question 33: Print Numbers Using Recursion

"""

Problem: Print numbers from 1 to n using recursion.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

"""

```

def print_numbers(n):
    if n > 0:
        print_numbers(n - 1)
        print(n, end=' ')

print("Question 33: Print Numbers 1 to n (n=10)")
print_numbers(10)
print("\n")

```

Question 34: Tower of Hanoi

"""

Problem: Solve Tower of Hanoi puzzle.

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

"""

```

def tower_of_hanoi(n, source, destination, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower_of_hanoi(n - 1, source, auxiliary, destination)
    print(f"Move disk {n} from {source} to {destination}")
    tower_of_hanoi(n - 1, auxiliary, destination, source)

print("Question 34: Tower of Hanoi (n=3)")
tower_of_hanoi(3, 'A', 'C', 'B')
print()

```

Question 35: Count Digits Using Recursion

"""

Problem: Count number of digits in a number using recursion.

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(d)$

"""

```
def count_digits(n):
```

```
    if n == 0:
```

```
        return 0
```

```
    return 1 + count_digits(n // 10)
```

```
def count_digits_v2(n):
```

```
    return len(str(abs(n))) if n != 0 else 1
```

```
print("Question 35: Count Digits")
```

```
test35 = 12345
```

```
print(f"Input: {test35}")
```

```
print(f"Output: {count_digits(test35)}")
```

```
print(f"Expected: 5\n")
```

===== SECTION 6: STRING MANIPULATION =====

Question 36: Convert to Uppercase

"""

Problem: Convert all characters in string to uppercase.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

"""

```
def to_uppercase(s):
```

```
    return s.upper()
```

```

def to_uppercase_manual(s):
    result = []
    for char in s:
        if 'a' <= char <= 'z':
            result.append(chr(ord(char) - 32))
        else:
            result.append(char)
    return "".join(result)

print("Question 36: Convert to Uppercase")
test36 = "Hello World!"
print(f"Input: '{test36}'")
print(f"Output: '{to_uppercase(test36)}'")
print(f"Expected: 'HELLO WORLD!'\n")

```

Question 37: Count Words in String

Problem: Count number of words in a string.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```

def count_words(s):
    return len(s.split())

def count_words_manual(s):
    count = 0
    in_word = False

    for char in s:
        if char.isspace():

```

```

        in_word = False
    elif not in_word:
        in_word = True
        count += 1

    return count

print("Question 37: Count Words")
test37 = "The quick brown fox jumps"
print(f"Input: '{test37}'")
print(f"Output: {count_words(test37)}")
print(f"Expected: 5\n")

```

Question 38: Remove Spaces from String

"""

Problem: Remove all spaces from a string.

* Time complexity: $O(n)$ where n is length of string

* space complexity: $O(n)$ for storing result

```

class Q38_RemoveSpaces {
    // Method 1: Using StringBuilder
    public static String removeSpaces(String str) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) != ' ') {
                result.append(str.charAt(i));
            }
        }
        return result.toString();
    }
}

```



```

}

// Method 2: Using replaceAll
public static String removeSpacesV2(String str) {
    return str.replaceAll("\\s+", "");
}

// Test Cases
public static void test() {
    System.out.println("=== Q38: Remove Spaces ===");
    System.out.println(removeSpaces("Hello World")); // HelloWorld
    System.out.println(removeSpaces(" Java Programming ")); // JavaProgramming
    System.out.println(removeSpaces("Infosys")); // Infosys
    System.out.println(removeSpaces("")); // (empty)
}
}

```

#QUESTION 39: Count Occurrences of Character

PROBLEM STATEMENT: Count how many times a specific character appears in a string.

* ALGORITHM:

1. Initialize count = 0
2. Loop through each character
3. If character matches target, increment count
4. Return count

Time complexity: $O(n)$

space complexity: $O(1)$

"""

```
class Q39_CountCharOccurrences {
    public static int countOccurrences(String str, char ch) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == ch) {
                count++;
            }
        }
        return count;
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q39: Count Occurrences ===");
        System.out.println(countOccurrences("programming", 'g')); // 2
        System.out.println(countOccurrences("hello", 'l')); // 2
        System.out.println(countOccurrences("java", 'z')); // 0
        System.out.println(countOccurrences("aaaaaa", 'a')); // 6
    }
}
```

QUESTION 40: String to Integer Conversion

"""

PROBLEM STATEMENT:

Convert a string representation of a number to an integer without using built-in functions.

* ALGORITHM:

1. Initialize result = 0
2. For each character, convert to digit: digit = ch - '0'
3. result = result * 10 + digit
4. Handle negative numbers if first char is '-'

* TIME COMPLEXITY: O(n)

* SPACE COMPLEXITY: O(1)

"""

```
class Q40_StringToInteger {
    public static int stringToInt(String str) {
        if (str == null || str.length() == 0) return 0;

        int result = 0;
        int sign = 1;
        int i = 0;

        // Handle negative sign
        if (str.charAt(0) == '-') {
            sign = -1;
            i = 1;
        }

        // Process each digit
```

```

    for (; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (ch < '0' || ch > '9') break;
        int digit = ch - '0';
        result = result * 10 + digit;
    }

    return result * sign;
}

// Test Cases
public static void test() {
    System.out.println("\n=== Q40: String to Integer ===");
    System.out.println(stringToInt("12345")); // 12345
    System.out.println(stringToInt("-678")); // -678
    System.out.println(stringToInt("0")); // 0
    System.out.println(stringToInt("999")); // 999
}
}

```

QUESTION 41: Toggle Case of String

* PROBLEM STATEMENT:

* Convert uppercase letters to lowercase and lowercase to uppercase in a string.

* ALGORITHM:

1. Traverse each character
2. If uppercase, convert to lowercase
3. If lowercase, convert to uppercase
4. Keep other characters as is

* Time complexity: $O(n)$

* Space complexity: $O(n)$

```
class Q41_ToggleCase {
    public static String toggleCase(String str) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (Character.isUpperCase(ch)) {
                result.append(Character.toLowerCase(ch));
            } else if (Character.isLowerCase(ch)) {
                result.append(Character.toUpperCase(ch));
            } else {
                result.append(ch);
            }
        }
        return result.toString();
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q41: Toggle Case ===");
        System.out.println(toggleCase("Hello World")); // hELLO wORLD
        System.out.println(toggleCase("Java123")); // jAVA123
        System.out.println(toggleCase("INFOSYS")); // infosys
    }
}
```

QUESTION 42: Check if String is Rotation

* PROBLEM STATEMENT: Check if one string is rotation of another.

ALGORITHM:

1. Check if lengths are equal
2. Concatenate str1 with itself
3. Check if str2 is substring of concatenated string

* Time complexity: $O(n)$

* Space complexity: $O(n)$

```
class Q42_StringRotation {
    public static boolean isRotation(String str1, String str2) {
        if (str1.length() != str2.length()) return false;
        String concatenated = str1 + str1;
        return concatenated.contains(str2);
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q42: String Rotation ===");
        System.out.println(isRotation("waterbottle", "erbottlewat")); // true
        System.out.println(isRotation("hello", "llohe")); // true
        System.out.println(isRotation("java", "avaj")); // false
        System.out.println(isRotation("abc", "bca")); // true
    }
}
```

// ===== SECTION 7: NUMBER PROBLEMS =====

QUESTION 43: Sum of Natural Numbers

* PROBLEM STATEMENT: Find sum of first n natural numbers.

* ALGORITHM:

Using formula: $\text{return } n * (n + 1) / 2$

* Time complexity: $O(1)$ using formula, $O(n)$ using loop

* Space complexity: $O(1)$

```
class Q43_SumOfNaturals {  
    // Method 1: Using formula (Efficient)  
    public static int sumUsingFormula(int n) {  
        return n * (n + 1) / 2;  
    }  
  
    // Method 2: Using loop  
    public static int sumUsingLoop(int n) {  
        int sum = 0;  
        for (int i = 1; i <= n; i++) {  
            sum += i;  
        }  
        return sum;  
    }  
  
    // Test Cases  
    public static void test() {  
        System.out.println("\n=== Q43: Sum of Natural Numbers ===");  
        System.out.println(sumUsingFormula(5)); // 15  
        System.out.println(sumUsingFormula(10)); // 55  
        System.out.println(sumUsingFormula(100)); // 5050  
        System.out.println(sumUsingFormula(1)); // 1  
    }  
}
```

```
}  
}
```

QUESTION 44: Print Prime Numbers in Range

* PROBLEM STATEMENT: Print all prime numbers between two given numbers.

* ALGORITHM:

1. Loop from start to end
2. For each number, check if it's prime
3. Prime check: test divisibility from 2 to $\sqrt{n}$

* TIME COMPLEXITY: $O(n * \sqrt{n})$

* SPACE COMPLEXITY: $O(1)$

```
class Q44_PrimesInRange {  
    public static void printPrimes(int start, int end) {  
        for (int num = start; num <= end; num++) {  
            if (isPrime(num)) {  
                System.out.print(num + " ");  
            }  
        }  
        System.out.println();  
    }  
}
```

```
private static boolean isPrime(int n) {  
    if (n <= 1) return false;  
    if (n == 2) return true;  
    if (n % 2 == 0) return false;  
    for (int i = 3; i <= Math.sqrt(n); i += 2) {
```



```

        if (n % i == 0) return false;
    }
    return true;
}

// Test Cases
public static void test() {
    System.out.println("\n=== Q44: Primes in Range ===");
    printPrimes(10, 30); // 11 13 17 19 23 29
    printPrimes(1, 20); // 2 3 5 7 11 13 17 19
    printPrimes(50, 60); // 53 59
}
}

```

// QUESTION 45: Automorphic Number

*** PROBLEM STATEMENT:** Check if a number is automorphic (its square ends with the number itself).

*** ALGORITHM:**

- * 1. Calculate square of number
- * 2. Extract last digits of square (same count as original number)
- * 3. Compare with original number

*** TIME COMPLEXITY:** $O(1)$

*** SPACE COMPLEXITY:** $O(1)$

```

class Q45_AutomorphicNumber {
    public static boolean isAutomorphic(int n) {
        long square = (long) n * n;
    }
}

```

```

while (n > 0) {
    if (n % 10 != square % 10) return false;
    n /= 10;
    square /= 10;
}
return true;
}

// Test Cases

public static void test() {
    System.out.println("\n=== Q45: Automorphic Number ===");
    System.out.println(isAutomorphic(5)); // true ( $5^2 = 25$ )
    System.out.println(isAutomorphic(6)); // true ( $6^2 = 36$ )
    System.out.println(isAutomorphic(25)); // true ( $25^2 = 625$ )
    System.out.println(isAutomorphic(76)); // true ( $76^2 = 5776$ )
    System.out.println(isAutomorphic(10)); // false
}
}

```

// QUESTION 46: Neon Number

* PROBLEM STATEMENT:

* Check if a number is a neon number (sum of digits of its square equals the number).

* ALGORITHM:

- * 1. Calculate square of number
- * 2. Find sum of digits of square
- * 3. Return true if sum equals original number

* TIME COMPLEXITY: $O(\log n)$

* SPACE COMPLEXITY: $O(1)$

```

class Q46_NeonNumber {
    public static boolean isNeon(int n) {
        int square = n * n;
        int sum = 0;
        while (square > 0) {
            sum += square % 10;
            square /= 10;
        }
        return sum == n;
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q46: Neon Number ===");
        System.out.println(isNeon(9)); // true (81 -> 8+1=9)
        System.out.println(isNeon(1)); // true (1 -> 1)
        System.out.println(isNeon(10)); // false
        System.out.println(isNeon(8)); // false
    }
}

```

#QUESTION 47: Abundant Number

* **PROBLEM STATEMENT:** Check if a number is abundant (sum of proper divisors > number).

* ALGORITHM:

* 1. Find all proper divisors (excluding the number itself)

* 2. Calculate sum of divisors

* 3. Return true if sum > number

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(1)$

```
class Q47_AbundantNumber {
    public static boolean isAbundant(int n) {
        int sum = 0;
        for (int i = 1; i <= n / 2; i++) {
            if (n % i == 0) {
                sum += i;
            }
        }
        return sum > n;
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q47: Abundant Number ===");
        System.out.println(isAbundant(12)); // true
        System.out.println(isAbundant(18)); // true
        System.out.println(isAbundant(20)); // true
        System.out.println(isAbundant(15)); // false
        System.out.println(isAbundant(28)); // false (perfect number)
    }
}
```

#QUESTION 48: Happy Number

* PROBLEM STATEMENT: Check if a number is happy (repeatedly summing squares of digits eventually gives 1).

* ALGORITHM:

- * 1. Use a set to detect cycles
- * 2. Repeatedly calculate sum of squares of digits
- * 3. If sum becomes 1, return true
- * 4. If we see a repeated sum (cycle), return false

* TIME COMPLEXITY: $O(\log n)$

* SPACE COMPLEXITY: $O(\log n)$ for the set

```
class Q48_HappyNumber {
    public static boolean isHappy(int n) {
        java.util.Set<Integer> seen = new java.util.HashSet<>();
        while (n != 1 && !seen.contains(n)) {
            seen.add(n);
            n = sumOfSquares(n);
        }
        return n == 1;
    }

    private static int sumOfSquares(int n) {
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum += digit * digit;
            n /= 10;
        }
    }
}
```

```

    }
    return sum;
}

// Test Cases
public static void test() {
    System.out.println("\n=== Q48: Happy Number ===");
    System.out.println(isHappy(19)); // true
    System.out.println(isHappy(7)); // true
    System.out.println(isHappy(2)); // false
    System.out.println(isHappy(1)); // true
}
}

```

// ===== SECTION 8: ARRAY OPERATIONS =====

QUESTION 49: Find Equilibrium Index

* PROBLEM STATEMENT: Find index where sum of elements on left equals sum on right.

* ALGORITHM:

- * 1. Calculate total sum of array
- * 2. Traverse array maintaining leftSum
- * 3. $\text{rightSum} = \text{totalSum} - \text{leftSum} - \text{arr}[i]$
- * 4. If $\text{leftSum} == \text{rightSum}$, return index

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(1)$

```

class Q49_EquilibriumIndex {
    public static int findEquilibrium(int[] arr) {

```

```

int totalSum = 0;
for (int num : arr) totalSum += num;

int leftSum = 0;
for (int i = 0; i < arr.length; i++) {
    int rightSum = totalSum - leftSum - arr[i];
    if (leftSum == rightSum) return i;
    leftSum += arr[i];
}
return -1;
}

// Test Cases

public static void test() {
    System.out.println("\n=== Q49: Equilibrium Index ===");
    System.out.println(findEquilibrium(new int[] {1, 3, 5, 2, 2})); // 2
    System.out.println(findEquilibrium(new int[] {1, 2, 3})); // -1
    System.out.println(findEquilibrium(new int[] {-7, 1, 5, 2, -4, 3, 0})); // 3
}
}

```

#QUESTION 50: Kadane's Algorithm (Max Subarray Sum)

* **PROBLEM STATEMENT:** Find maximum sum of contiguous subarray.

* **ALGORITHM:**

- * 1. Initialize maxSum = arr[0], currentSum = arr[0]
- * 2. For each element:
 - * - currentSum = max(element, currentSum + element)
 - * - maxSum = max(maxSum, currentSum)

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(1)$

```
class Q50_MaxSubarraySum {
    public static int kadane(int[] arr) {
        int maxSum = arr[0];
        int currentSum = arr[0];

        for (int i = 1; i < arr.length; i++) {
            currentSum = Math.max(arr[i], currentSum + arr[i]);
            maxSum = Math.max(maxSum, currentSum);
        }

        return maxSum;
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q50: Max Subarray Sum ===");
        System.out.println(kadane(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4})); // 6
        System.out.println(kadane(new int[]{1, 2, 3, 4, 5})); // 15
        System.out.println(kadane(new int[]{-1, -2, -3, -4})); // -1
    }
}
```

#QUESTION 51: Move All Zeros to End

* PROBLEM STATEMENT: Move all zeros to the end while maintaining order of non-zero elements.

* ALGORITHM:

- * 1. Use pointer 'index' for non-zero elements
- * 2. Traverse array, if element is non-zero, place at 'index' position
- * 3. Fill remaining positions with zeros

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(1)$

```
class Q51_MoveZerosToEnd {
    public static void moveZeros(int[] arr) {
        int index = 0;

        // Move all non-zero elements to front
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[index++] = arr[i];
            }
        }

        // Fill remaining with zeros
        while (index < arr.length) {
            arr[index++] = 0;
        }
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q51: Move Zeros to End ===");
        int[] arr1 = {0, 1, 0, 3, 12};
        moveZeros(arr1);
        System.out.println(java.util.Arrays.toString(arr1)); // [1, 3, 12, 0, 0]
    }
}
```

```

        int[] arr2 = {0, 0, 1};

        moveZeros(arr2);

        System.out.println(java.util.Arrays.toString(arr2)); // [1, 0, 0]
    }
}

```

#QUESTION 52: Rearrange Array in Alternating Fashion

* PROBLEM STATEMENT:

* Rearrange array in alternating positive and negative numbers.

* ALGORITHM:

* 1. Create two lists for positive and negative numbers

* 2. Merge alternately into result array

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(n)$

```

class Q52_AlternatePositiveNegative {
    public static int[] rearrange(int[] arr) {
        java.util.List<Integer> pos = new java.util.ArrayList<>();
        java.util.List<Integer> neg = new java.util.ArrayList<>();

        for (int num : arr) {
            if (num >= 0) pos.add(num);
            else neg.add(num);
        }

        int[] result = new int[arr.length];
    }
}

```

```

int i = 0, p = 0, n = 0;

while (p < pos.size() && n < neg.size()) {
    result[i++] = neg.get(n++);
    result[i++] = pos.get(p++);
}

while (p < pos.size()) result[i++] = pos.get(p++);
while (n < neg.size()) result[i++] = neg.get(n++);

return result;
}

// Test Cases
public static void test() {
    System.out.println("\n=== Q52: Alternate Positive Negative ===");
    System.out.println(java.util.Arrays.toString(
        rearrange(new int[] {1, 2, 3, -4, -1, 4})
    ));
}
}

```

#QUESTION 53: Find Leaders in Array

* **PROBLEM STATEMENT:** Find all leaders in array (element greater than all elements to its right).

* ALGORITHM:

- * 1. Start from rightmost element (always a leader)
- * 2. Traverse right to left
- * 3. If current element > max seen so far, it's a leader
- * 4. Update max

* TIME COMPLEXITY: $O(n)$

* SPACE COMPLEXITY: $O(n)$ for storing leaders

```
class Q53_LeadersInArray {  
    public static java.util.List<Integer> findLeaders(int[] arr) {  
        java.util.List<Integer> leaders = new java.util.ArrayList<>();  
        int maxRight = arr[arr.length - 1];  
        leaders.add(maxRight);  
  
        for (int i = arr.length - 2; i >= 0; i--) {  
            if (arr[i] > maxRight) {  
                leaders.add(arr[i]);  
                maxRight = arr[i];  
            }  
        }  
  
        java.util.Collections.reverse(leaders);  
        return leaders;  
    }  
  
    // Test Cases  
    public static void test() {  
        System.out.println("\n=== Q53: Leaders in Array ===");  
    }  
}
```

```

        System.out.println(findLeaders(new int[]{16, 17, 4, 3, 5, 2})); // [17, 5, 2]
        System.out.println(findLeaders(new int[]{1, 2, 3, 4, 5})); // [5]
        System.out.println(findLeaders(new int[]{5, 4, 3, 2, 1})); // [5, 4, 3, 2, 1]
    }
}

// ===== SECTION 9: MATHEMATICAL PROBLEMS =====

```

#QUESTION 54: Check if Number is Spy Number

* **PROBLEM STATEMENT:** Check if sum of digits equals product of digits (Spy Number).

* **ALGORITHM:**

- * 1. Initialize sum = 0, product = 1
- * 2. Extract each digit
- * 3. Add to sum, multiply to product
- * 4. Compare sum and product

* **TIME COMPLEXITY:** $O(\log n)$

* **SPACE COMPLEXITY:** $O(1)$

```

class Q54_SpyNumber {
    public static boolean isSpy(int n) {
        int sum = 0, product = 1;
        while (n > 0) {
            int digit = n % 10;
            sum += digit;
            product *= digit;
            n /= 10;
        }
    }
}

```

```
        return sum == product;
    }

    // Test Cases
    public static void test() {
        System.out.println("\n=== Q54: Spy Number ===");
        System.out.println(isSpy(1124)); // true
        System.out.println(isSpy(123)); // true (1+2+3=6, 1*2*3=6)
        System.out.println(isSpy(132)); // false
        System.out.println(isSpy(22)); // false
    }
}
```



About the Author

SYNTAX ERROR | Abhishek Rathor

Helping students crack their dream placements! 🚀

Connect with me:

- 📷 Instagram: [@code.abhii07](https://www.instagram.com/code.abhii07)
 - 📺 YouTube: [SYNTAX ERROR](https://www.youtube.com/channel/UCv33333333333333333333)
-



Youve Got This! All The Best!



"Success is where preparation meets opportunity. You've prepared well - now go seize your opportunity!"